



Universidad Tecnológica Nacional
Facultad Regional Córdoba

Algoritmos y Estructuras de Datos

Ficha Nro. 12

Arreglos Unidimensionales

Prof. Ing. Corso Cynthia

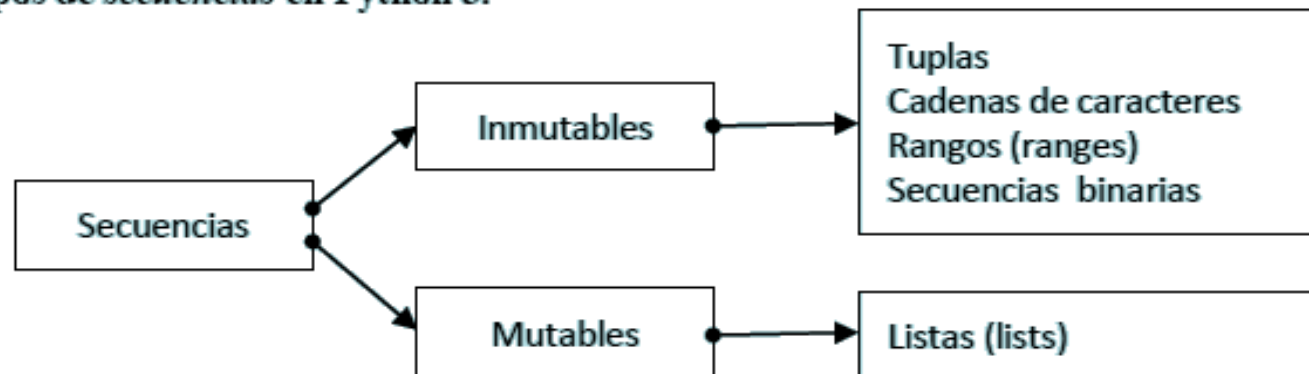
Temario

- Introducción
- Arreglos Unidimensionales en Python
 - Creación
 - Incorporación de elementos.
 - Acceso a componentes individuales de una variable tipo List.
 - Formas de recorridos con listas.
 - Subrangos.Corte de índices.

Introducción

- Estructura de datos: es una variable que puede contener varios valores a la vez.
- Python provee ciertas estructuras de datos en forma de **secuencias**.
- La ventaja de usar estructuras de datos para el desarrollo programas, es la posibilidad de manejar un volumen significativo de datos y/o resultados.

Tipos de secuencias en Python 3.

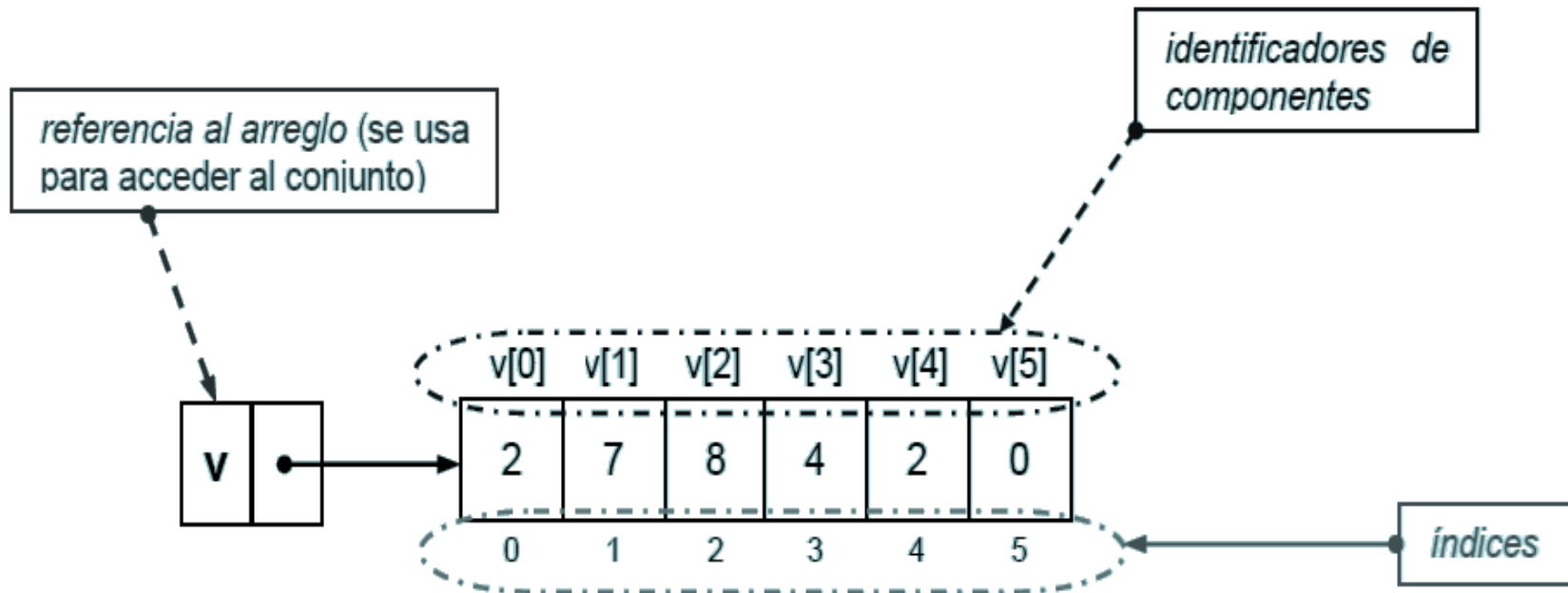


Arreglos unidimensionales en Python

- **Arreglo unidimensional**: es una estructura de datos básica que almacena un conjunto de valores, de manera tal que cada componente individual es identificado por un número denominado **índice**.
- El uso de índices permite el acceso, uso y/o modificación de cada componente en forma individual.
- La cantidad de índices que se requieren para acceder a un elemento individual, se llama **dimensión del arreglo**.
- Los arreglos unidimensionales se denominan así, porque requieren solo un índice para acceder a un componente.

Arreglos unidimensionales: Estructura

Figura 1: Esquema básico de un arreglo unidimensional.



Arreglos unidimensionales: Conceptos

- Se podría pensar que las secuencias ya estudiadas (cadenas de caracteres, tuplas, rangos), pueden ser consideradas como arreglos unidimensionales.
- Sin embargo este tipo de secuencias ya estudiadas tienen una restricción, son de tipo **inmutable**. Es por ello que las secuencias de tipo inmutable, no representan operativamente arreglos unidimensionales.
- Python provee un tipo de secuencia designado como la **lista o (list)** , que permite a un programador gestionar un arreglo en forma completa.
- Por su similitud que existe entre el concepto de arreglo unidimensional y el concepto de vector en Algebra, suelen también llamarse **vectores**.

Arreglos unidimensionales: Conceptos

- En Python es posible la definición de un arreglo (lista) que puede contener valores de distintos tipos.

```
datos = ['Juan', 35, True, 5678.56]
```

- En el caso de esta definición de arreglo (lista) el programador deberá tener en cuenta en que posición almacenó un determinado tipo de dato para evitar un procesamiento incorrecto.

Creación de un arreglo unidimensional (I)

- Para crear un arreglo unidimensional (lista) “v” vacío, se debe definir una variable de tipo `list` en cualquiera de las dos forma que se muestra a continuación:

```
v = [] o  
v = list()
```

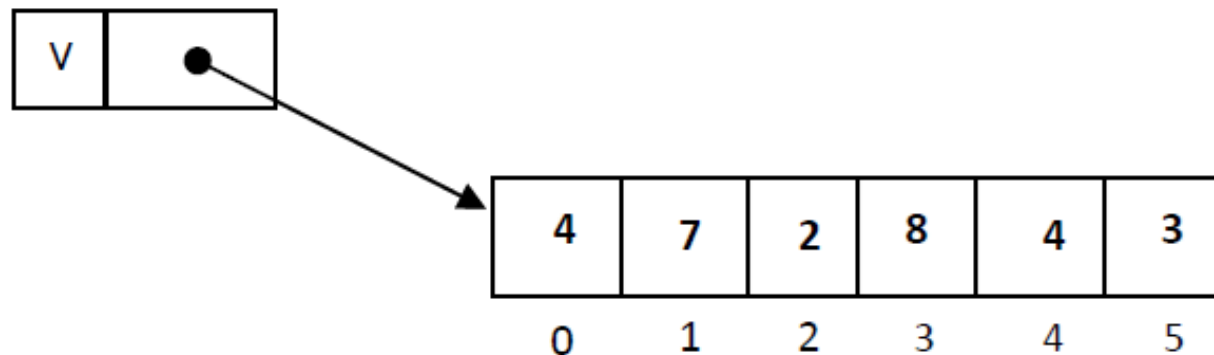
- Si se conoce los valores de antemano los valores que se desea almacenar, se puede crear **enumerando sus componentes**.

```
v = [4, 7, 2, 8, 4, 3]
```


Creación de un arreglo unidimensional (II)

- La última instrucción anterior crea un arreglo de seis componentes (por enumeración), inicializa cada casilla de ese arreglo con los valores enumerados entre los corchetes, y retorna la dirección la dirección del arreglo.

Figura 2: Un arreglo con seis componentes de tipo int.



Visualización de un arreglo unidimensional

- Al igual que las cadenas y tuplas, el contenido completo de una variable `list` puede mostrarse directamente en la consola estándar mediante una simple invocación a la función `print()`.
- La siguiente secuencia de instrucciones muestra contenido de un arreglo:

```
v = [4, 7, 2, 8, 4, 3]
print('Contenido del arreglo:')
print(v)
```

- Producirá la siguiente salida:

```
Contenido del arreglo: [4, 7, 2, 8, 4, 3]
```

Creación de un arreglo unidimensional (III)

- Si se necesita crear un arreglo de tamaño conocido, y preasignado con valores iguales (por ejemplo ceros) se puede hacer usando el **operador *** (**multiplicación**). Ejemplo: crear un arreglo con “n” elementos (n = 4)

```
# Definición arreglo notas con 4 componentes.  
# Cada componente se inicializa con cero.  
notas = 4 * [0]  
print('El arreglo notas:', notas)  
#Visualiza por consola  
[0, 0, 0, 0]
```

- Creado el arreglo, se usa la referencia que lo apunta para acceder a sus componentes, ya sea para **consultarlos o modificarlos**, colocando a la derecha un **par de corchetes y el índice del casillero** que se desea acceder.

Acceso y modificación de componentes

Operaciones posibles con una variable de tipo `list`.

```
v = [4, 10, 2, 8, 4, 3]
```

```
# asigna el valor 4 en la casilla 3
```

```
v[3] = 4
```

```
# suma 1 al valor contenido en la casilla 1
```

```
v[1] += 1
```

```
# muestra en consola el valor de la casilla 0
```

```
print(v[0])
```

```
# asigna en la casilla 4 el valor de la casilla #  
1 menos el de la 0
```

```
v[4] = v[1] - v[0]
```

```
# carga por teclado un entero y lo asigna en la #  
casilla 5
```

```
v[5] = int(input('Valor: '))
```

```
# resta 8 al valor de la casilla 2
```

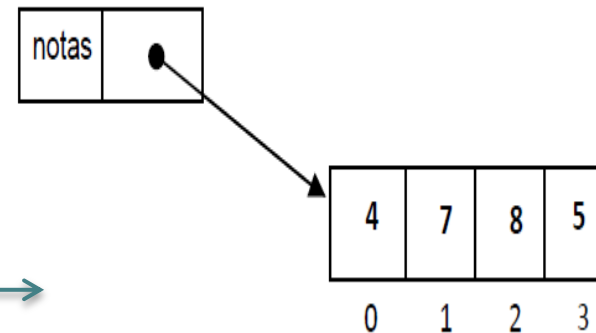
```
v[2] = v[2] - 8
```

Carga de un arreglo unidimensional (I)

- Incorporación de elementos: si el arreglo se creó vacío se puede agregar elementos por medio del método `append()`.

```
notas = []  
notas.append(4)  
notas.append(7)  
notas.append(8)  
notas.append(5)  
print('Notas:', notas)
```

Figura 3: Efecto de la función *append()* al crear una variable de tipo *list*.



La función `append()` agrega el contenido de la variable que es tomado como parámetro a una lista, pero lo hace de forma que ese valor se agrega al **final del arreglo**.

Carga de un arreglo unidimensional (II)

- En el caso de que el arreglo se cree con un número fijo de componentes, se puede usar un ciclo **for** y se usa el índice para acceder a cada casilla y proceder a la carga de elementos.
- El siguiente esquema muestra la forma de hacer la carga de cuatro valores por teclado en el arreglo.

```
notas1 = 4 * [0] # Vector con 4 componentes.  
for i in range(4):  
    # Carga las notas por teclado  
    notas1[i] = int(input('Cargue nota:'))
```

Carga de un arreglo unidimensional (III)

- Otra alternativa que logra el mismo efecto del script anterior, es usar la función `len()` para conocer el tamaño o cantidad de elementos que tiene una variable de tipo list.

```
notas1 = 4 * [0] # Vector con 4 componentes.  
for i in range(len(notas1)):  
    # Carga las notas por teclado  
    notas1[i] = int(input('Cargue nota:'))
```

Procesamiento secuencial de un arreglo

- Si el arreglo está ya creado, se puede procesar a cada uno de los componentes iterando sobre ellos, o sobre sus índices.
- Para ello se utiliza un ciclo **for** lo que permite un recorrido completo del mismo.

Ejemplo: Obtener el promedio de notas del arreglo unidimensional.

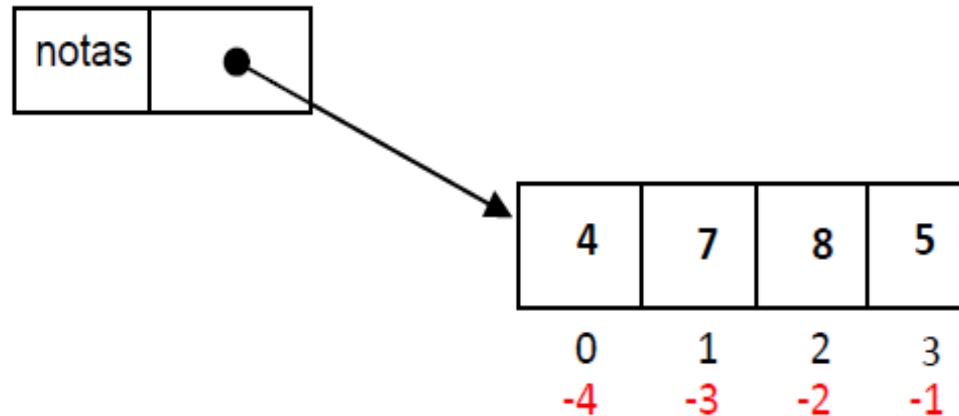
```
suma = 0
notas1 = 4 * [0]
n = len(notas1)
for i in range(n):
    suma += notas1[i]
promedio = suma / n
print('El promedio es:', promedio)
```


Índices negativos en un arreglo unidimensional

- En Python una variable de tipo `list` es **dinámica**, es decir que se pueden agregar, quitar elementos en cualquier posición o incluso cambiar el tipo de sus componentes.
- Los elementos individuales de una variable de tipo `list` pueden accederse mediante **índices**.
- En Python se pueden usar **índices negativos** con cualquier tipo de secuencia (listas, tuplas, cadenas). Considerando que el índice `-1` representa el último elemento, el `-2` el penúltimo elemento y así sucesivamente.

Índices negativos en un arreglo unidimensional

Figura 4: Esquema de índices para una secuencia de cualquier tipo en Python.



Índices negativos en un arreglo unidimensional

- Creación de un arreglo unidimensional y diferentes posibilidades de recorridos de izquierda a derecha y viceversa (uso de índices negativos).

```
# crear una lista con n números del 1 al n...
```

```
n = 10
```

```
numeros = []
```

```
for i in range(1, n+1):
```

```
    numeros.append(i)
```

```
print("Lista original: ", numeros)
```

```
# recorre de izquierda a derecha y mostrar los
```

```
# números pares...
```

```
print("Valores pares contenidos en la lista: ")
```

```
for i in range(len(numeros)):
```

```
    if numeros[i] % 2 == 0:
```

```
        print(numeros[i])
```

Índices negativos en un arreglo unidimensional

- Recorrido del arreglo unidimensional de derecha a izquierda.

```
# (Continuación del script anterior)
# recorrer de derecha a izquierda y mostrar
# todos los números (uso de índices negativos)...

print("\nContenido de la lista, en forma invertida: ")
for i in range(-1, (-len(numeros) - 1), -1):
    print(numeros[i])
```

Modificación y eliminación de elementos en un arreglo unidimensional

- En Python una variable de tipo list son de naturaleza **mutable**. El siguiente ejemplo cambia el valor del elemento en la posición 2 por el valor -1.

```
numeros = [3, 5, 8, 6]
numeros [2] = -1
print(numeros)
# muestra: [3, 5, -1, 6]
```

- Si se desea eliminar un elemento particular, se puede recurrir la instrucción **del**.

```
numeros = [3, 5, 8, 6, 9, 2]
del numeros[3]
print(numeros) # muestra: [3, 5, 8, 9, 2]
```

Subrangos o Corte de índices

- Todo tipo de secuencia en Python permite acceder a un subrango de casilleros “rebanando” o “cortando” sus índices.

```
numeros = [4, 2, 3, 4, 7]
nueva = numeros [1:4]
print("\nNueva lista: ", nueva)
# muestra: [2, 3, 4]
```

- La siguiente instrucción también usa un corte de índices pero esta vez para copiar una lista.

```
numeros = [2, 4, 6, 5]
copia = numeros[:] # copia toda la lista
print(copia) # muestra: [2, 4, 6, 5]
```

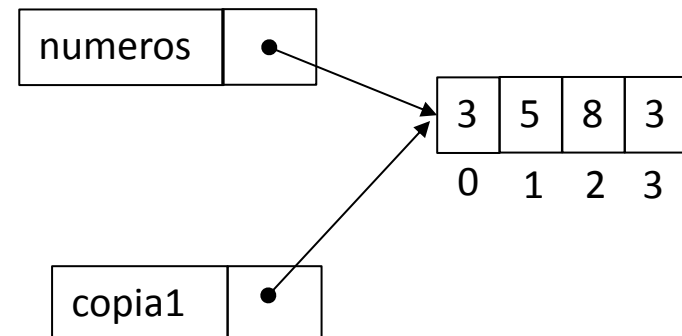
Subrangos o Cortes de índices

- Notar la diferencia entre **asignar entre sí** dos variables de tipo list o usar **corte de índices** para copiar un arreglo completo.

```
numeros = [3, 5, 8, 3]
# una copia a nivel de referencias (copia
# superficial)...
copia1 = numeros # ambas variables apuntan a la
# misma lista...
numeros[2] = -1
print("Original: ", numeros)
print("Copia 1: ", copia1)
```

```
numeros: [3, 5, -1, 3]
copia1: [3, 5, -1, 3]
```

Se produce la siguiente salida.



Subrangos o Cortes de índices

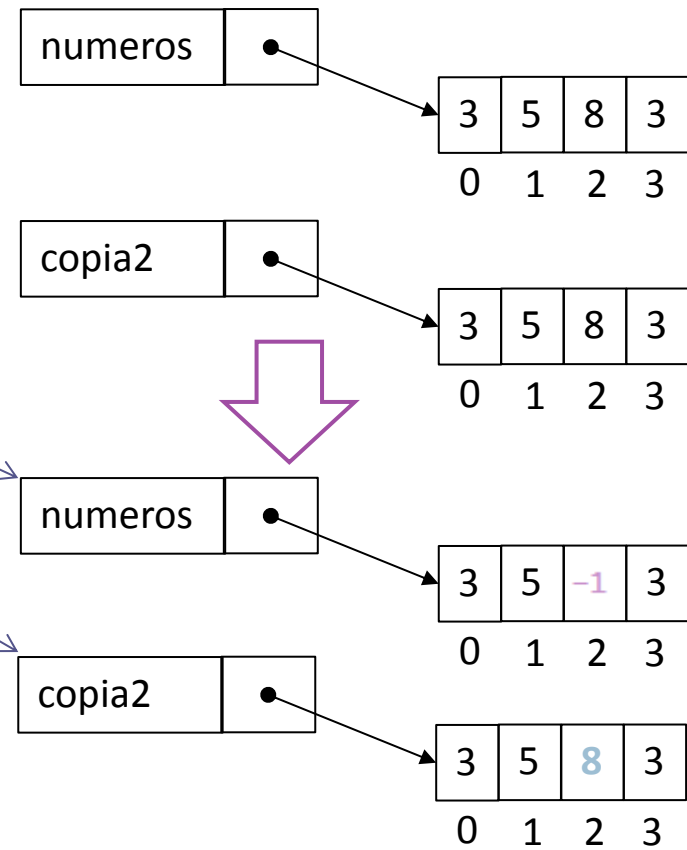
- En el caso de usar **corte de índices** se esta recuperando una copia de elementos. El siguiente script no copia las referencias, sino que crea un segundo arreglo cuyos elementos son iguales a los del primero.

```
numeros = [3, 5, 8, 3]
# una copia a nivel valores...
copia2 = numeros[:]
```

```
numeros[2] = -1
print("Original: ", numeros)
print("Copia 2: ", copia2)
```

```
numeros: [3, 5, -1, 3]
copia2: [3, 5, 8, 3]
```

Se produce la siguiente salida.



Muchas Gracias por la atención!!

Cualquier duda pueden contactarme por mensajería interna del Aula Virtual.